



CABINET CHEAT SHEET

Power Glove Vision Quick Reference

Current cabinet addresses, services, controls, and maintenance commands.

2026 EDITION / IAIN BENNETT / MIT LICENSE

Use this guide to install, pair, check, and operate your PowerGlove Vision system. Replace `UNO-Q-NAME.local` with your UNO Q hostname and `RETROPIE-NAME.local` with your Raspberry Pi hostname. Each command section identifies the machine on which to run it. Keep passwords and pairing tokens out of this document.

Your installation

Item	Value
UNO Q network address	<code>UNO-Q-NAME.local</code>
RetroPie network address	<code>RETROPIE-NAME.local</code>
UNO Q App Lab application	PowerGlove Vision
UNO Q application directory	<code>/home/arduino/ArduinoApps/powerglove-vision</code>
Camera	UVC-compatible USB camera; select <code>auto</code> in Setup
Startup profile	Choose in Setup

Prefer `.local` names in bookmarks and settings. If a name does not resolve, check the device's current IP address in your router and use that address temporarily. A router reservation prevents the fallback address from changing.

Browser URLs

Open these pages on a computer or phone connected to the same trusted network.

Page	Address
Dashboard: camera and controller output	Open Dashboard
Learn: practice and tune gestures	Open Learn
Games: edit game mappings	Open Games
Help: manuals and live cabinet reference	Open Help
Setup: connection and startup settings	Open Setup
Secure Setup: pairing	Open secure Setup
Status: diagnostic readings	Open status
Camera stream	Open camera stream
Project repository	PowerGlove Vision on GitHub

The links above contain example hostnames. Replace them in the browser's address bar. The live **Help > This cabinet** page builds links using the UNO Q address you used to open it.

Install and deploy over Wi-Fi

For a first installation, follow these sections in order, then pair the devices and complete the [Installation Guide's gameplay checks](#). All flags are explained in the [command reference](#).

Download and package on your computer

- 1 Install Arduino App Lab on your development computer. Run `command -v git python3 bash rsync zip` and install any missing tools.
- 2 In a macOS or Linux terminal, choose your projects folder and run the commands below. They download the current `dev` branch and build its UNO Q installation ZIP.
- 3 Confirm that verification reports **App Lab installation ZIP verified**.

```
SH
git clone --branch dev https://github.com/mathan416/PowerGlove-Vision.git
cd PowerGlove-Vision
scripts/build-app-lab-package.sh
python3 scripts/verify-app-lab-package.py
```

Prepare the UNO Q

- 1 Connect the UNO Q by USB and complete its setup in App Lab. Join the same network as RetroPie and record the board's hostname.
- 2 Import `output/app-lab/PowerGlove-Vision-Uno-Q.zip` from your computer's checkout. Open **PowerGlove Vision** and select **Run** to transfer and start the app and matrix sketch.
- 3 Connect the camera through the powered USB hub. Follow the [Installation Guide](#) if you need help with the initial board setup.

Open a terminal on the UNO Q, or connect from your computer:

```
SH
ssh arduino@UNO-Q-NAME.local
```

Run these commands **on the UNO Q** after importing and running the app once:

```
SH
cd /home/arduino/ArduinoApps/powerglove-vision
sudo python3 scripts/setup-machine.py uno-q
```

This installs host support for local names and the shutdown helper, sets the app to start at boot, and restarts it. Review every **FAIL** or **ACTION** result. The installer requires the application directory shown above. Run `exit` after setup to leave the UNO Q terminal. Check Dashboard and Learn before pairing.

Install the Raspberry Pi receiver before pairing

The Raspberry Pi needs the receiver, pairing command, game registry, and launch hooks before you can pair it with the UNO Q. Run the following in a terminal **on the Raspberry Pi running RetroPie**. You can use a local terminal or SSH with your RetroPie account.

For a new installation, download the same `dev` branch used on the UNO Q:

```
SH
sudo apt update
sudo apt install -y git
cd ~
git clone --branch dev https://github.com/mathan416/PowerGlove-Vision.git
cd PowerGlove-Vision
```

If you already have a checkout, open that directory instead of cloning again. Then install the RetroPie components, substituting your UNO Q hostname:

```
SH
sudo python3 scripts/setup-machine.py retropie --peer UNO-Q-NAME.local
```

The installer preserves existing tokens and settings, installs the receiver and pairing commands under [/opt/powerglove/bin/](#), and adds the game-launch hooks. It also installs the controller mapping and the 45-second startup timer. An **ACTION** result asking you to pair or verify gameplay is expected on first installation. Correct any **FAIL** result before continuing to pairing.

For an existing installation, `--peer` does not replace the saved UNO Q address. If that address has changed, update [/etc/powerglove/launcher.json](#) on RetroPie.

Update the UNO Q from your computer

Complete the [SSH key setup](#) first. From the full project checkout **on your development computer**, verify access and deploy:

```
SH
ssh -o BatchMode=yes arduino@UNO-Q-NAME.local hostname
scripts/deploy-uno-q-wifi.sh arduino@UNO-Q-NAME.local
```

The deployment preserves the UNO Q's private [data/](#) directory and restarts the application. It updates the UNO Q only. To update RetroPie, update its source checkout and rerun the RetroPie installer above; it preserves local settings.

The UNO Q installer includes the shutdown helper. To update or repair that helper separately, run this from your development computer's project checkout:

```
SH
scripts/install-uno-q-shutdown-helper.sh arduino@UNO-Q-NAME.local
```

The terminal prompts for the UNO Q account password if needed. The helper requests a Linux halt; the tested board restarts afterward. See the shutdown limitation below.

Pair your RetroPie

Complete both machine installations above, then use the one-time-code method:

- 1 On RetroPie, run `sudo /opt/powerglove/bin/powerglove-pair` and leave it running. Its code expires after two minutes.
- 2 In your browser, open <https://UNO-Q-NAME.local:8443/setup> using your UNO Q's actual hostname.
- 3 Enter your RetroPie hostname and the 20-character code printed by the pairing command.

- 4 Select **Prepare one-time code**. Compare the matrix [ID](#) with the beginning of the browser certificate's SHA-256 fingerprint.
- 5 If they match, select the certificate confirmation checkbox, enter the six-digit PIN displayed after [PN](#) on the matrix, and select **Complete pairing**.
- 6 On RetroPie, run `sudo systemctl status powerglove-receiver.service` and confirm that the receiver is active.

Password pairing is available on the same secure page when RetroPie accepts SSH password login. Select **Prepare password pairing**, complete the same physical certificate check, and then enter your RetroPie account password. The password is used for one SSH operation and is not stored.

Quick health checks

From your computer, open the status URL above or run:

```
SH
curl -sS http://UNO-Q-NAME.local:8088/status
```

Once you have selected an active profile and completed calibration, the status readings should show the following while your hand is visible:

```
JSON
{
  "camera_available": true,
  "worker_running": true,
  "detected": true,
  "calibrated": true
}
```

With **Gestures off**, an inactive camera is normal. Select a profile on Dashboard or open Learn to check tracking.

Run these checks **on RetroPie**:

```
SH
sudo systemctl status powerglove-receiver.service
sudo systemctl status powerglove-receiver.timer
sudo journalctl -u powerglove-receiver.service -n 100 --no-pager
grep -A8 -B2 'PowerGlove Vision' /proc/bus/input/devices
```

The virtual controller appears after the first authenticated packet. Select **Start controller** on Dashboard when you are ready to send input.

Run the local software tests

This is a developer check. Run it from the **root of the full Git checkout**, where the `src/` and `tests/` directories are present:

```
SH
PYTHONPATH=src python3 -m unittest discover -s tests -v
```

The command is correct for macOS and Linux. `PYTHONPATH=src` lets the tests import the local source without installing the package. The tests do not require a camera, MediaPipe, or either physical device, but some open temporary local network listeners. The App Lab ZIP and the installed receiver files do not include the full test suite. Successful completion ends with **OK**; these tests do not replace a check of the controls in a running game.

Camera troubleshooting

Connect the camera to the **UNO Q** through a powered USB hub. A camera attached to your computer is not available to the UNO Q application.

Open a terminal on the UNO Q, using SSH if necessary:

```
SH
ssh arduino@UNO-Q-NAME.local
```

Run the following **on the UNO Q** to see each Linux video device and its name:

```
SH
for device in /sys/class/video4linux/video*; do
[ -r "$device/name" ] || continue
printf '%s: ' "/dev/${device##*/}"
cat "$device/name"
done
```

Entries named `qcom-venus-encoder` or `qcom-venus-decoder` are the board's internal video codecs, not the webcam. Look for an additional device whose name matches your USB camera. If only codec entries appear, or no entries appear, Linux has not exposed a webcam video device. Check hub power, reconnect the camera, and run the command again.

You can also check for persistent USB-camera links:

```
SH
ls -l /dev/v4l/by-id/
```

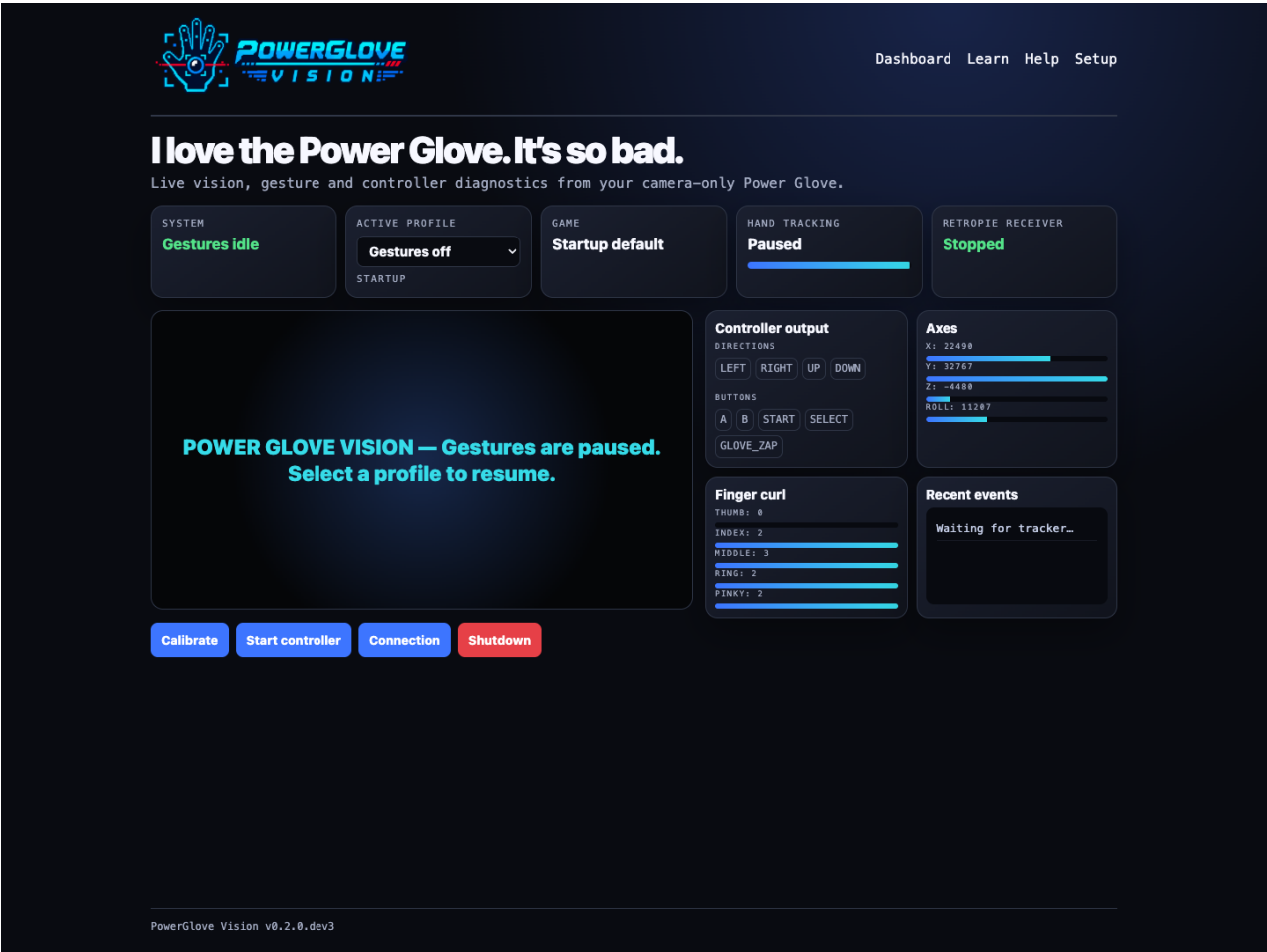
That directory may be absent on some systems; its absence alone does not prove that the camera is missing. Use the device-name listing above as well.

Return to Dashboard, select an active profile, and watch for the camera view. The app retries camera initialization automatically. Keep **Camera** set to `auto` unless you have identified a specific capture device to select.

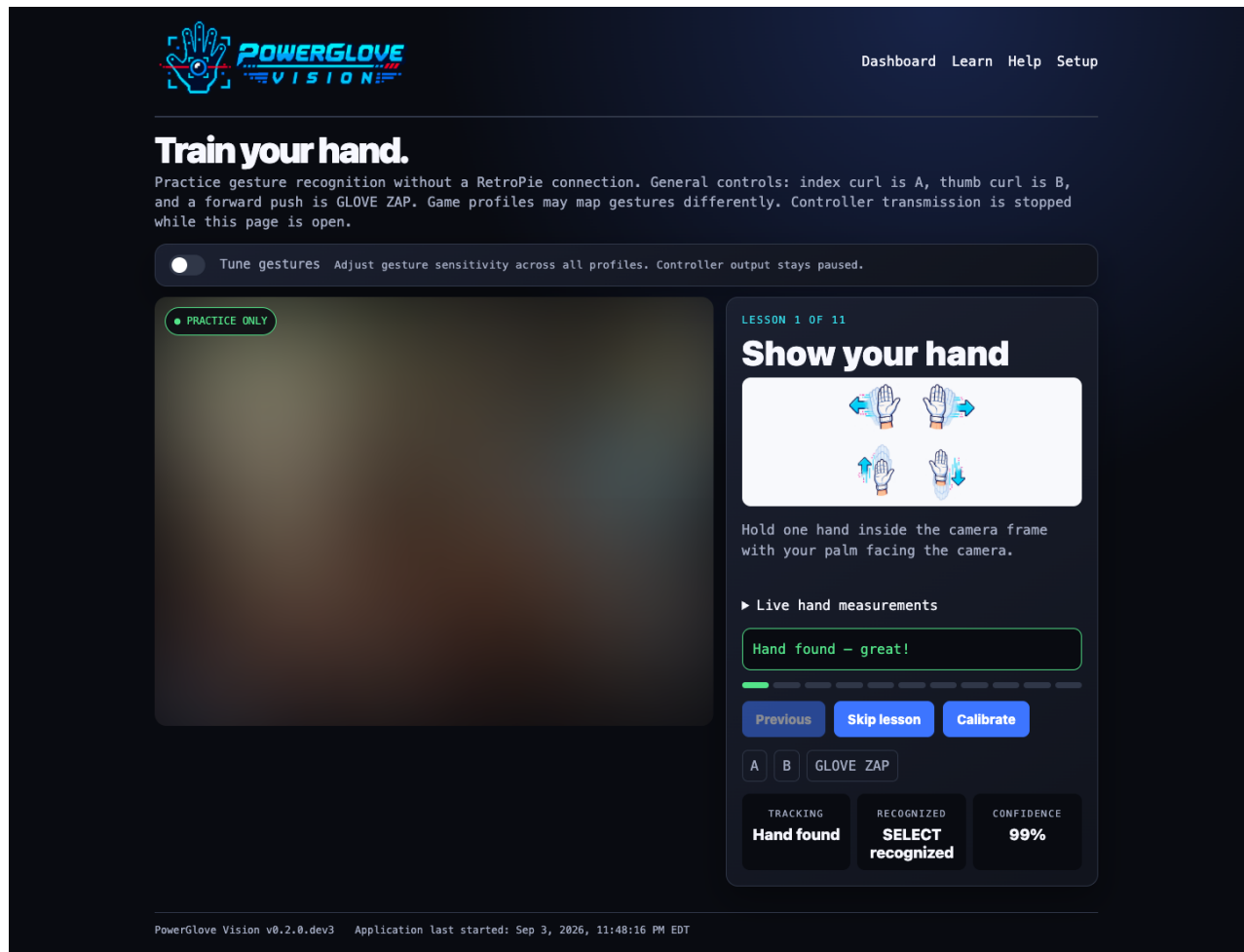
Website screenshots

These are reference screenshots, not live views. Open the browser URLs above to see your own camera and controller status. Camera imagery is blurred for privacy. These screenshots were refreshed on September 4, 2026.

Dashboard

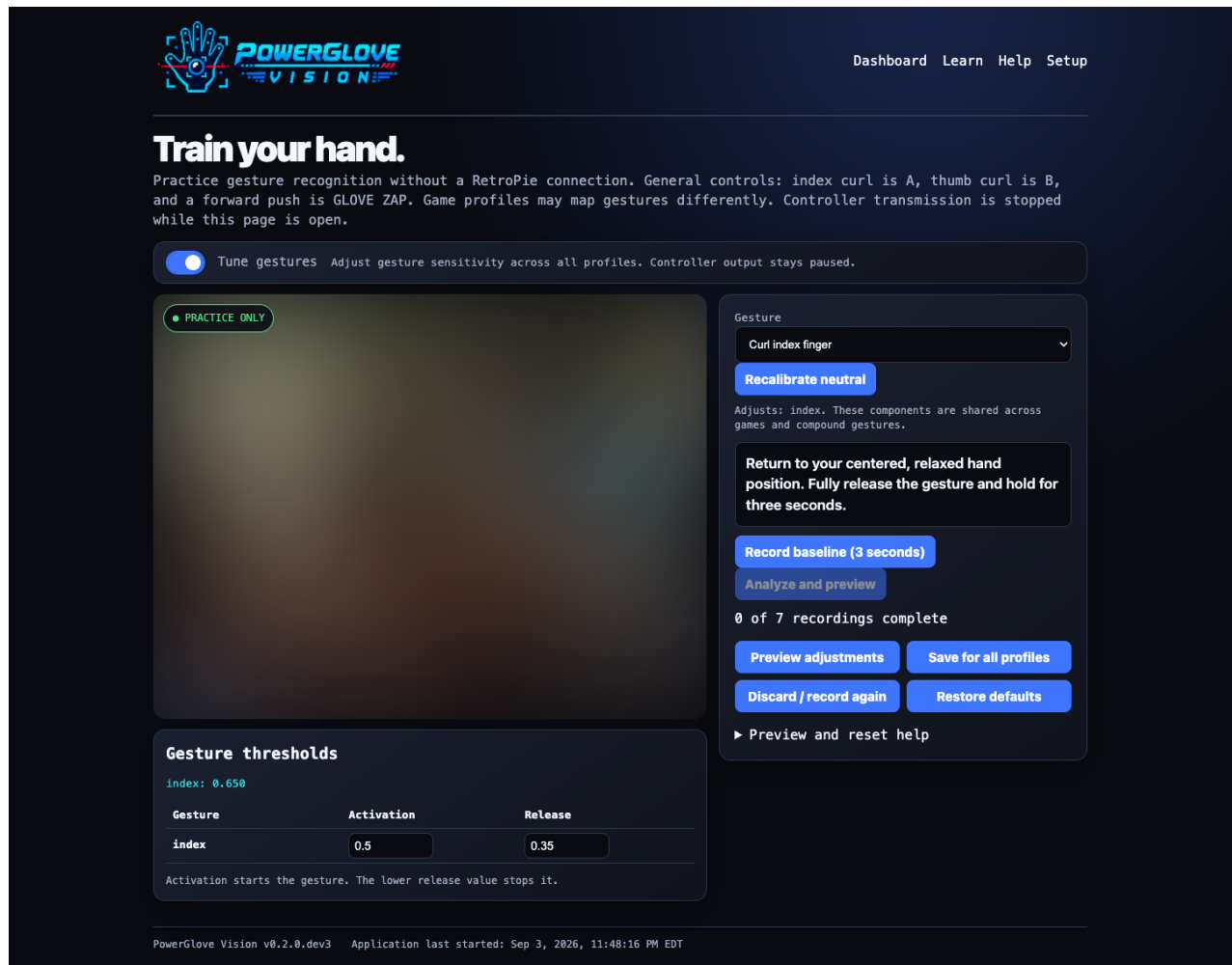


Dashboard showing Gestures off and controller diagnostics



Learn page with its camera imagery blurred for privacy

Tune gestures



Tune mode with thresholds below the blurred camera

The matrix shows **T** while tuning. Instructions and recording controls sit beside the camera on a wide screen; Activation and Release are underneath the camera.

Setup

POWERGLOVE VISION

Dashboard Learn Help Setup

Let's get connected.

Tell PowerGlove Vision where your RetroPie console lives. Settings are saved on this UNO Q and the private pairing token is never shown.

RetroPie console name: retropieconsole.local Controller port: 55355 Startup profile: Gestures off Tracking aid: Bare hand

Camera: auto

☐ Generate a new private pairing token

Save & restart tracker Test console name Start controller Shutdown

PAIRING

Private token configured — confirm pairing on RetroPie

Your matching token remains in `data/device.json` and must also be installed at `/etc/powerglove/token` on RetroPie.

ADDRESS

`/setup`

Bookmark this page at your UNO Q's `.local:8088` address.

Pair with RetroPie

Pairing is disabled over HTTP. Open [the secure setup page](#).

RetroPie address: retropieconsole.local RetroPie username: pi RetroPie password: UNO Q approval PIN: Shown on the LED matrix

☐ I compared the browser certificate fingerprint with the matrix ID

Prepare password pairing

► Advanced: pair without a RetroPie password

Setup page for connection settings and pairing

The Setup screenshot shows the HTTP page. Open secure Setup on port 8443 to pair.

Games within Setup

Games

Edit the game mappings installed on your RetroPie. Saving affects the next game launch.

Use the exact ROM filename, including its extension: `Joust (USA).7z` and `Joust (USA).nes` need separate entries. Matching ignores letter case. Only NES and Famicom launches use these mappings.

► Available profile identifiers

Game mappings JSON

```
{
  "games": {
    "Bad Street Brawler (USA).nes": "bad_street_brawler",
    "Bad Street Brawler (USA).zip": "bad_street_brawler",
    "Bad Street Brawler (USA).7z": "bad_street_brawler",
    "Super Glove Ball (USA).nes": "super_glove_ball",
    "Super Glove Ball (USA).zip": "super_glove_ball",
    "Super Glove Ball (USA).7z": "super_glove_ball",
    "Joust (USA).nes": "program_b",
    "Gyruss (USA).nes": "program_c",
    "Defender II (USA).nes": "program_e",
    "Sesame Street 1-2-3 (USA).nes": "program_f",
    "Gun.Smoke (USA).nes": "program_g",
    "Knight Rider (USA).nes": "program_i",
    "Joust (USA).zip": "program_b",
    "Joust (USA).7z": "program_b",
    "Gyruss (USA).zip": "program_c",
    "Gyruss (USA).7z": "program_c",
    "Defender II (USA).zip": "program_e",
    "Defender II (USA).7z": "program_e",
    "Sesame Street 1-2-3 (USA).zip": "program_f",
    "Sesame Street 1-2-3 (USA).7z": "program_f",
    "Gun.Smoke (USA).zip": "program_g"
```

Validate

Format

Save

Reload

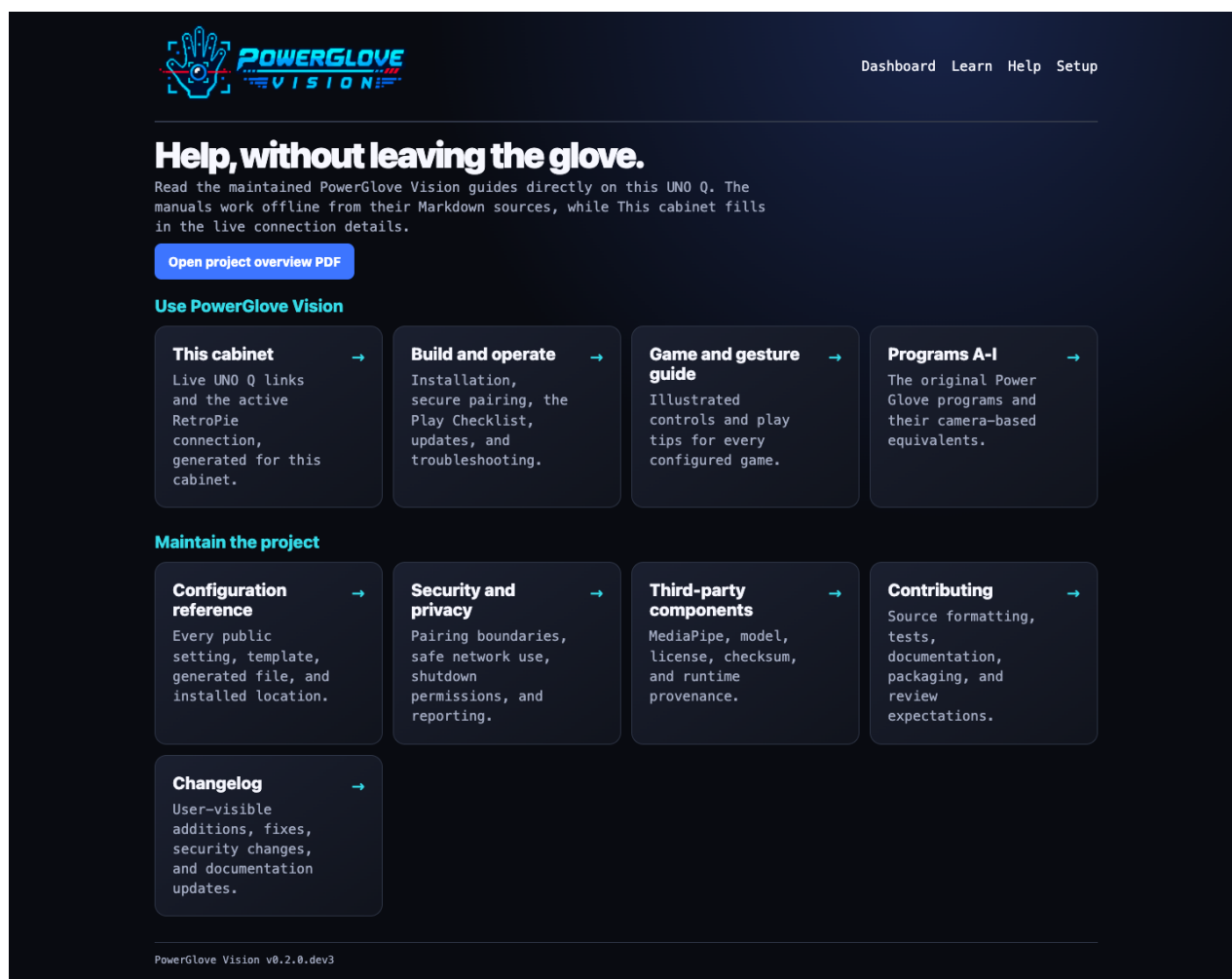
Download backup

Restore previous save

Installed mappings loaded.

Games editor below the pairing section

Scroll down Setup to edit mappings. Saving affects the next game launch.



Help page linking to the manuals and printable editions

Choose a profile

Use **Active profile** on Dashboard to choose the controls for your current session. Use **Startup profile** on Setup to choose the profile the app loads when it starts. Available choices are:

- Bad Street Brawler
- Super Glove Ball
- Programs A-I
- Gestures off

Wait for the camera to start

When you select an active profile or open Learn, the app may display **Starting camera and gesture tracking** while it opens the camera and loads the tracker. The elapsed time shows how long initialization has been running. Wait for the live camera view before calibrating.

Gestures off closes the camera. Learn temporarily opens it for practice and suppresses game input. Leaving Learn restores the selected profile.

RetroPie launch hooks select the registered profile when a recognized game starts. If you launch an unregistered game or a game for a system other than NES or Famicom, the launch hook selects **Gestures off**. Ending a game also turns gestures off.

Start with these reusable profiles

Program	Useful for	Controls
A	Pinball and games with two independent actions	Index curl sends A; thumb curl sends Up; wrist roll sends B. Pulling back toggles combined flippers. Ordinary hand movement does not control the D-pad.
D	A reversed-direction challenge	Hand movement sends the opposite direction. Thumb curl sends A; index curl sends B.
H	General NES and Famicom experiments	Hand movement controls the D-pad. Index curl pulses A; thumb curl pulses B. Avoid this profile when a game needs a continuously held action button.

Try a profile in a game

- 1 Launch an unregistered NES or Famicom game. PowerGlove Vision should show **Gestures off**.
- 2 Open Dashboard and choose **A: Pinball**, **D: Challenge**, **H: General**, or another profile.
- 3 Wait for the camera view. Hold your open hand in your comfortable resting position. This is your **neutral position**: the position the app treats as the center for movement.
- 4 If a direction remains active while your hand is at rest, select **Calibrate** and hold still. Also recalibrate after moving the camera or changing your playing position.
- 5 Select **Start controller** and test movement, actions, Start, and Select in the game.
- 6 Select **Stop controller** before adjusting the camera or testing another mapping.

This Dashboard choice is temporary. It does not change the saved startup profile or the game's automatic profile assignment.

Make a game use your chosen profile automatically

Once a profile works well, register the game on **RetroPie**. The launch hook reads `/etc/powerglove/games.json` to choose the profile each time a game starts.

- 1 Find the game file in your RetroPie ROM folder, usually `~/RetroPie/roms/nas/`. Record its complete filename, including the extension. For example, `/home/pi/RetroPie/roms/nas/My Game (USA).zip` has the filename `My Game (USA).zip`. Use the archive filename when launching an archive, not the filename inside it.
- 2 Open **Setup -> Games** on the UNO Q website and select **Download backup**.
- 3 Edit the loaded JSON in the Games section.
- 4 Add the filename and your chosen profile inside the existing `games` object. Keep all existing entries, separate entries with commas, and leave no comma after the last entry.

- 5 Select **Validate**, then **Save**. Wait for verified save confirmation and restart the game. **Restore previous save** reverses the last saved edit.

This example shows the required structure. Replace the example filename with your actual filename and merge the entry into your existing file:

```
JSON
{
  "games": {
    "My Game (USA).zip": "program_h"
  }
}
```

For a manual file edit outside the website, check syntax on RetroPie:

```
SH
python3 -m json.tool /etc/powerglove/games.json >/dev/null
```

If the command finishes without reporting an error, the JSON syntax is valid; it does not verify that the filename or profile is correct. Matching ignores letter case but otherwise requires the same filename, including spaces, punctuation, and [.nes](#), [.zip](#), or [.7z](#). Confirm the selected profile on Dashboard after restarting the game.

See the [Gameplay Guide](#) for game-specific instructions and the [Programs A-I manual](#) for all reusable mappings.

A **profile queued** launch message means the UNO Q accepted the request for processing. Confirm the active profile and game name on Dashboard. For timeouts, see [Check a queued profile change](#); the UNO Q must publish UDP [55356](#), and the registry must match the exact archive filename.

Tune a gesture

- 1 Open Learn, show your hand, and switch on **Tune gestures**.
- 2 Select a gesture and record your relaxed baseline.
- 3 Record three repetitions of the gesture and its release, following the prompts.
- 4 Select **Analyze and preview**, then try the suggested values.
- 5 Select **Save for all profiles**, or discard the preview. **Restore defaults** resets the selected components.

Controller delivery stays paused during tuning. Start it explicitly from Dashboard when ready to play. See [Tune gesture sensitivity](#) for guidance on noisy samples, neutral calibration, and shared finger thresholds.

Service and configuration reference

Item	Location or name
RetroPie virtual controller	PowerGlove Vision
RetroPie pairing token	/etc/powerglove/token
RetroPie game registry	/etc/powerglove/games.json
RetroPie connection settings	/etc/powerglove/launcher.json
Receiver service	powerglove-receiver.service
Receiver startup timer	powerglove-receiver.timer ; starts 45 seconds after boot
UNO Q shutdown watcher	powerglove-system-shutdown.path
UNO Q shutdown action	powerglove-system-shutdown.service ; requests a Linux halt
UNO Q readiness marker	/home/arduino/ArduinoApps/powerglove-vision/data/.shutdown-enabled
UNO Q boot rule that creates the marker	/etc/tmpfiles.d/powerglove-system-shutdown.conf ; installed from uno-q/powerglove-system-shutdown.conf

The boot rule creates the readiness marker; it does not initiate shutdown or prove that shutdown has completed. The watcher responds to a separate [data/shutdown-request](#) file created after you confirm **Shutdown** in the browser. Update the rule and its matching service files together using the helper installation command under **Install and deploy over Wi-Fi**.

Verify the helper **on the UNO Q** without requesting a shutdown:

```
SH
systemctl is-enabled powerglove-system-shutdown.path
systemctl is-active powerglove-system-shutdown.path
ls -l /home/arduino/ArduinoApps/powerglove-vision/data/.shutdown-enabled
```

Expect [enabled](#), [active](#), and an existing marker file. On RetroPie, keep the receiver timer enabled and the receiver service disabled for direct boot activation. The timer starts the service after EmulationStation initializes.

Network ports

Port	Direction	Purpose
TCP 8088	Browser -> UNO Q	Dashboard, Learn, Help, Setup, status, and camera stream
TCP 8443	Browser -> UNO Q	Secure Setup and pairing
UDP 55355	UNO Q -> RetroPie	Controller-state packets
UDP 55356	RetroPie -> UNO Q	Profile requests and acknowledgements
TCP 55357	UNO Q -> RetroPie	Temporary one-time-code pairing server

Keep these ports on your trusted local network. Do not expose them to the internet.

Saved calibration and startup

Calibration records your resting hand position, apparent size, and wrist angle in the UNO Q's [data/calibration.json](#). It survives profile changes, Learn sessions, and restarts. Include it in private backups. Recalibrate when your physical setup changes or the resting hand position produces unwanted movement. The camera overlay's **Right** or **Left** label identifies the hand; its score is confidence in that identification, not confidence in a movement command.

Keep one PowerGlove Vision installation active in App Lab and set it as the default startup app. OpenCV and MediaPipe preload in the background while the website is available. **Gestures off** keeps the camera closed; select an active profile or open Learn to begin capture. An early request waits for preloading to finish. Controller transmission starts stopped, so select **Start controller** when ready to play.

With preloading complete, the first activation after a tested reboot took 1.21 seconds; actual times vary. For a slow start, inspect the [startup stage logs](#). If the camera disappears after reboot, check `lsusb` and `/dev/v4l/by-id/` on the UNO Q and reconnect the camera or hub if it is missing.

Known limitation: UNO Q restarts after Shutdown

Stop controller leaves Linux and the website running. **Shutdown** requests a graceful Linux halt. The tested UNO Q automatically restarts after halt, both with a powered hub and with a direct Mac USB connection. A disappearing website, matrix animation, or fixed waiting period does not confirm that power can safely be removed. See the [Installation Guide](#) for the recorded investigation and shutdown guidance.